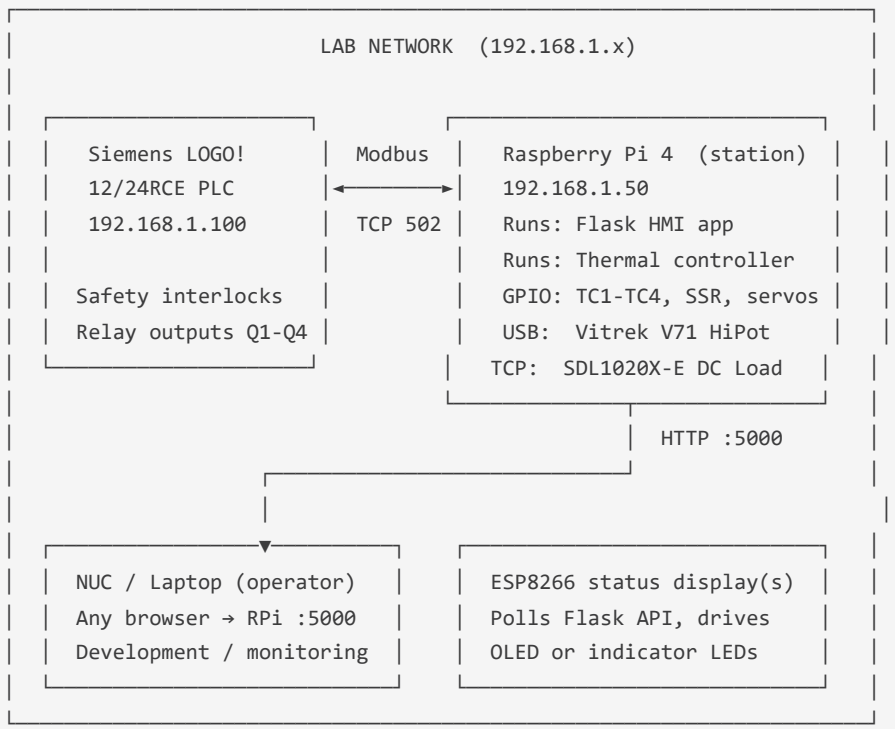
Test Station HMI Architecture

> **Print-ready PDF:** `docs/pdf/hmi-architecture.pdf`

This document describes the recommended hardware layout for the Juniper Automated Test Station and defines the role of each computing device.

Architecture Overview



Device Roles

Raspberry Pi 4 — Primary Station Computer

Recommended model: Raspberry Pi 4B, 4 GB RAM, running Raspberry Pi OS Lite (64-bit)

The RPi is the central brain of the test station. It runs the Flask web application (`app.py`) and hosts the thermal controller hardware interface. Everything else on the network connects to or reports to the RPi.

What runs on the RPi:

Component	Role
`app.py` (Flask)	Web HMI server — serves the operator UI on port 5000
`plc/thermal_controller.py`	PID heater loop, thermocouple reads, servo vent control
`plc/logo_driver.py`	Modbus TCP client — reads/writes the LOGO! PLC
`sdl1020x_driver.py`	SCPI TCP client — controls the SDL1020X-E DC load
`v71_driver.py`	USB HID driver — controls the Vitrek V71 HiPot tester
`database.py`	SQLite sensor log at 1 Hz — stores all test and thermal data
`excel_export.py`	Generates Juniper-branded `.xlsx` test reports

Physical connections from RPi:

RPi connector	Goes to	Protocol
GPIO SPI0 (pins 19, 21, 23)	4× MAX31855 thermocouple amplifiers	SPI
GPIO 12 (pin 32)	Fotek SSR-40DA heater control	PWM @ 10 Hz
GPIO 13 (pin 33)	Vent servo A	PWM @ 50 Hz
GPIO 18 (pin 12)	Vent servo B	PWM @ 50 Hz
Ethernet	Lab network switch	100/1000 Mbps
USB-A	Vitrek V71 HiPot tester	USB HID → UART

What to buy (if RPi not already on hand):

Item	Qty	Notes
Raspberry Pi 4B 4GB	1	Pi 5 also works; Pi 3B+ is marginal
SD card 32 GB+ Class 10	1	Samsung EVO or better
USB-C 5V/3A power supply	1	Official Pi PSU recommended
DIN rail mount case for Pi	1	e.g. RasPi DIN mount carrier — keeps it tidy in the enclosure
Ethernet patch cable	1	Short run from Pi to switch/router

Siemens LOGO! 12/24RCE — Safety PLC

The LOGO! handles all hardware safety interlocks. It monitors the E-stop, door interlock, and overtemp thermostat, and gates every relay output through those conditions regardless of software state. The RPi writes "software enable" requests via Modbus; the LOGO! ladder decides whether the relay actually energises.

The LOGO! does not run the test logic — that lives in the Flask app. The LOGO! is always-on infrastructure: it powers up with the station, runs its FBD program continuously, and the RPi connects to it when the web app starts.

See `docs/logo-soft-comfort-guide.md` for how to program the LOGO!, and `docs/plc-ladder-logic.md` for the full FBD specification.

NUC / Laptop — Operator Workstation

The NUC (or any PC on the lab network) is the operator's screen. It does not run any test software — it simply opens a browser and navigates to `http://<RPi-IP>:5000`. The full HMI renders in the browser.

The NUC is also used for development: editing code, committing to git, running the `push.ps1` pipeline, and reviewing test data in Excel.

If a dedicated operator screen is desired, a touchscreen monitor connected to the RPi's HDMI output with Chromium in kiosk mode is a clean option.

ESP8266 Devices — Supplementary Status Displays / Bridge

The WeMos D1 Minis and LoLin NodeMCU v3 in the kit are best used as low-cost, low-power status nodes. Three useful roles:

Role 1: OLED Status Display (recommended first build)

A WeMos D1 Mini with a 0.96" SSD1306 I²C OLED mounts at the front of the test chamber enclosure. It polls the Flask `/api/status` endpoint over WiFi every 2 seconds and shows: - Current chamber temperature (TC1_AMBIENT) - DUT surface temperature (TC2_DUT) - Active test / profile name - Alarm state (large text if fault)

Additional BOM items:

Item	Qty	Approx $	Notes
WeMos D1 Mini (ESP8266)	1	$4	Already in kit
SSD1306 OLED 0.96" I²C 128×64	1	$3	4-pin I²C version
Dupont wires F-F 4×	4	—	SDA, SCL, 3.3V, GND
Small enclosure / panel mount	1	$5	Mount to chamber door

Firmware sketch location (to be created): `esp8266/status-display/status-display.ino`

Role 2: WiFi Bridge for LOGO! Modbus (optional)

If running a long Ethernet cable from the RPi to the DIN rail is inconvenient, a NodeMCU v3 can act as a transparent Modbus TCP↔WiFi bridge (using the `ModbusRTU2TCP` bridge pattern). The RPi talks to the NodeMCU over WiFi; the NodeMCU forwards to the LOGO! via a short local

Ethernet run. This adds latency and a failure point — only worth doing if cabling is genuinely difficult.

Role 3: Alarm Annunciator (simple)

An ESP8266 polling `/api/plc/io` can drive a buzzer and an RGB LED for at-a-glance fault indication across the room — useful in a noisy lab where the LOGO!'s alarm beacon might not be heard.

Recommended Setup Order

1. Flash Raspberry Pi OS Lite (64-bit) to SD card.
2. Configure static IP `192.168.1.50` and enable SSH.
3. Clone the repo: `git clone https://github.com/jasonjuniper/vitrek-v71-controller.git`
4. Install dependencies: `pip install -r requirements.txt --break-system-packages`
5. Configure `plc/rig_config.json` to match your wiring (IPs, pin numbers).
6. Program the LOGO! PLC using `docs/logo-soft-comfort-guide.md`.
7. Wire the thermal rig per `docs/wiring-guide.md`.
8. Start the Flask app: `python app.py` (add a systemd service for auto-start on boot).
9. Open `http://192.168.1.50:5000` from the NUC to verify the HMI loads.
10. Build the ESP8266 status display using the firmware in `esp8266/status-display/`.
